Phase 1: Tree Implementation as a Conflict-Free Replicated Data Type (CRDT)

CS 403/534 - Distributed Systems
Course Project, Spring 2025-2026

April 22, 2026

Deadline: April 27, 23:55 (Turkish Time)

Abstract

In the first phase of your course project, you will implement a CRDT which incorporates a *tree* as its data structure and a *Move* operation as its only operation in Python as a library, and use that library in a main program, also written in Python, which you are going to complete. You will be using ZeroMQ sockets with the Publish-Subscribe pattern for communication through message passing between multiple replicas, and Lamport clocks as the logical clock for the replicas.

1 Description

1.1 Tree CRDT

The Tree CRDT is an operation-based CRDT proposed by Kleppmann et al. (2021) with distributed file systems in mind as a possible use-case.

The tree data structure held by the CRDT is defined as a set of edges such that an edge between a parent node and a child node is represented by a triple as follows:

$$(p, m, c)$$

where:

- p is the unique ID for the parent node (`None` if the child is the root node),
- m contains the metadata associated with the child node (see *main.py* for an example structure that you can use),
- c is the ID of the child node, which should be globally unique.

As the third field c has to be unique for each edge in a tree, i.e., each node has at most one parent, you can also consider the above definition to be a representation for <u>the node whose ID is the value of c</u>.

1.2 The `Move` Operation

The only operation supported by the Tree CRDT is the `Move` operation, through which one can set the parent and the metadata associated with a node to the values they provide for the operation, whose description you can find in the paper by Kleppmann et al. as follows:

$$\text{Move} \quad t \quad p \quad m \quad c$$

$$\text{tree}' = \{(p', m', c') \in \text{tree} : c' \neq c\} \cup \{(p, m, c)\}$$

where:

- t: A globally unique and totally ordered timestamp (in the case of Phase 1 of the course project, you will use **Lamport timestamps**),
- p: The globally unique ID of the new parent for the node,
- m: The metadata to be associated with the child node,
- c: The globally unique ID of the (child) node to be moved.

Some further details on the `Move` operation:

- Applying a `Move` operation for a child node that did not exist prior to the operation is equivalent to creating the node; thus, it is not an edge case, but rather a usual one. If the child node exists before in the tree, the move operation overwrites the old parent with the new one.
- As the data structure considered here is a tree, you cannot create a cycle with `Move` operations: for instance, a node cannot be moved under a node which is among its descendants. You should be checking for such circumstances in your implementations, and ignoring such operations from occurring at all.

In Figure 1, a sequence of `Move` operations applied on a tree, T, with an associated starting Lamport timestamp $t = 0$ is demonstrated in order to describe the different cases in which a `Move` operation can be utilized. The first operation, Move 1 None m_0 0, creates the root node of the tree, with a non-existent parent, indicated with `None`, and an ID of 0. It can also be observed from the operation that the Lamport timestamp related to the tree has been incremented while the operation is generated, which is the reason why the t field of the `Move` operation has a value of 1. The next two operations, Move 2 0 m_1 1 and Move 3 0 m_2 2, create two child nodes under the root node, with the respective IDs 1 and 2, and metadata m_1 and m_2.

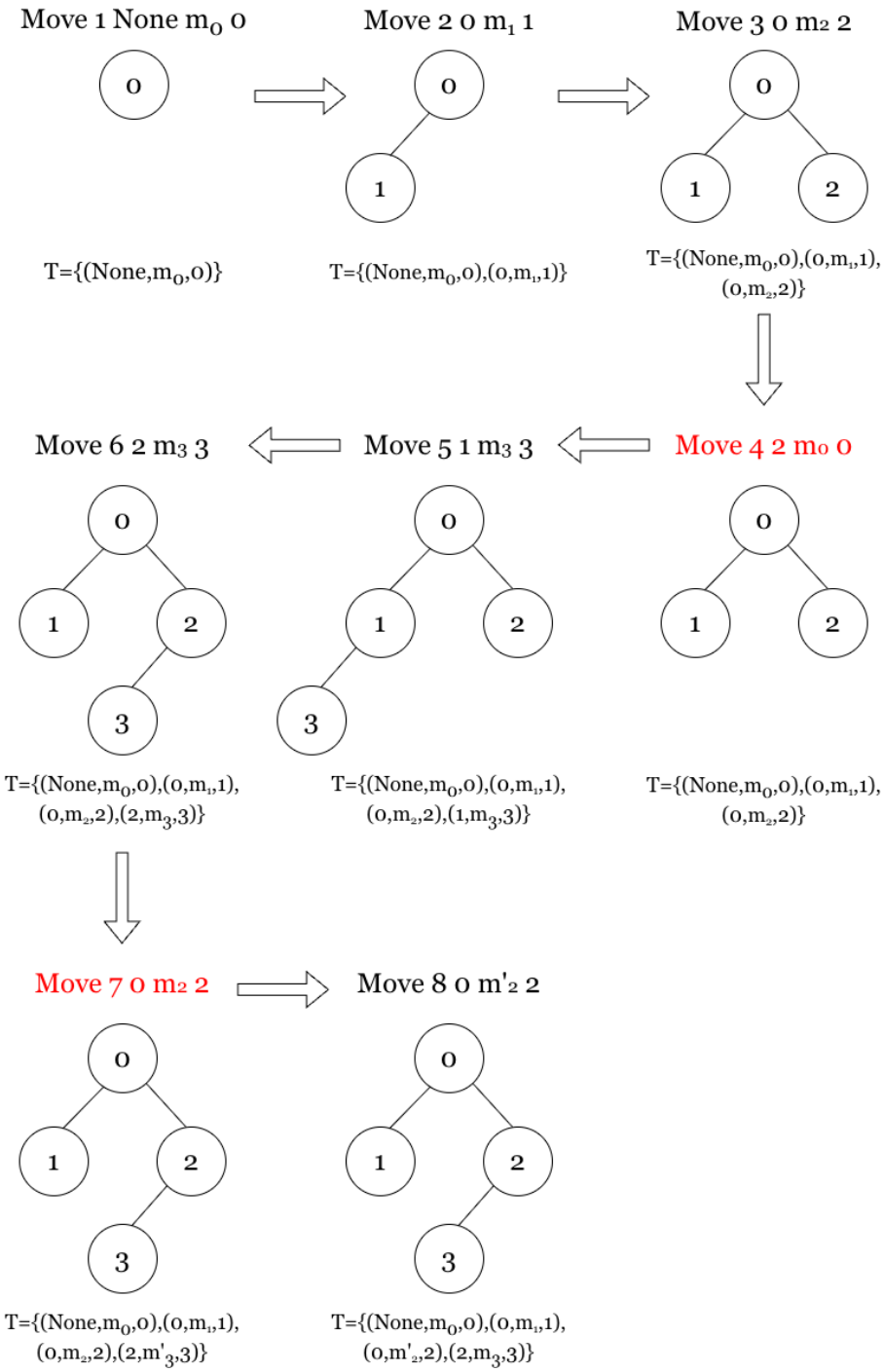

Figure 1: A sequence of `Move` operations applied on a tree, showing the creation, move, and metadata updates of the nodes; the operations written in red are ignored.

However, the operation Move 4 2 m_0 0 is the first ignored operation, indicated in red in the figure, because adding the edge $(2, m_0, 0)$ to the tree T, in which the edge $(0, m_2, 2)$ already exists, would cause a cycle between the nodes 0 and 2.

The subsequent operations shown in Figure 1 involve the creation of the node 3 with the operation Move 5 1 m_3 3, and the moving of the node from under node 1 to becoming a child of node 2 with Move 6 2 m_3 3.

The operation that is attempted to be applied after the aforementioned ones is the second operation to be ignored in the figure, as the operation Move 7 0 m_2 2 tries to add the edge $(0, m_2, 2)$ to the tree T, while the same edge is already present within the set representation of T. The last operation, Move 8 0 m_2' 2, in contrast, is successfully applied because it updates the metadata associated with the child node, node 2, from m_2 to m_2', where you can assume that $m_2' \neq m_2$, by removing the edge $(0, m_2, 2)$ from T and adding $(0, m_2', 2)$ instead.

1.3 The *Undo-Do-Redo* Sequence for the `Move` Operation

Since the messages are propagated asynchronously while implementing CRDTs, there may be certain situations in which you need to execute an operation with timestamp t, but you had already applied operations with timestamp values greater than t. In those cases, you will need to apply a sequence of <u>undo</u>, <u>do</u>, and <u>redo</u> actions for certain operations:

- You must <u>undo</u> the effects of operations whose timestamp is $t' > t$,
- <u>do</u> (apply) the operation with timestamp t afterwards,
- and <u>redo</u> the operations that you have previously undone.

In order to facilitate the undo-do-redo sequence described above, operations should be ordered by (t, i), where t is the timestamp and i is the ID of the replica which generated the operation. A tuple (t', i') can be called a <u>predecessor</u> of the tuple (t, i) ($(t', i') < (t, i)$) if $t < t'$ or $t' = t \wedge i' < i$, and a <u>successor</u> of the tuple (t, i) ($(t', i') > (t, i)$) if $t > t'$ or $t' = t \wedge i' > i$. The tuples (t, i) and (t', i') are <u>equivalent</u> in order if $t' = t \wedge i' = i$.

Note: See the material for LAB 5 - Conflict-Free Replicated Data Types (CRDTs) on SUCourse+ in order to get more information regarding CRDTs.

2 Description

2.1 Structure for the Replicas in the System

The first requirement for the replicas in the system is that each replica should run in its own process with a particular function to execute, which you can perform using the `multiprocessing` module in Python as will be described later.

The second requirement is that in each replica process, there should exist two threads: The main thread should primarily be responsible for generating `Move` operations, locally applying such operations, and broadcasting them to

other replicas, until the replica reaches a predefined maximum timestamp (see *README.md*), as it is going to be explained in the next subsection, whereas the second thread should act as a listener for the `Move` operations coming from other replicas, applying those operations on the receiving replica.

2.2 Communication Protocol Between the Replicas

Communication between the replicas is going to handled through message passing between ZeroMQ sockets with the Publish-Subscribe (**PUB-SUB**) pattern. Each replica has to have two *addresses*, each of which will correspond to a thread of the replica.

The detailed protocol for communication between replicas in the system is as follows:

- Each thread has to have two sockets: The main thread will have the *MOVE-PUB* and *ACK-SUB* sockets, while the listener thread will own the *MOVE-SUB* and *ACK-PUB* sockets for the replica.
- The *MOVE-PUB* socket will be bound to the address for the main thread, and the *ACK-PUB* socket will be bound to the address for the listener thread.
- The *MOVE-SUB* socket of a replica will be connected to the main thread addresses of other replicas, and the *ACK-SUB* socket of a replica will be connected to the listener thread addresses of other replicas.
- The main thread of a replica is going to broadcast messages regarding the details of the `Move` operations it locally generated and applied to the "MOVE" topic, until it reaches a predefined maximum Lamport timestamp (see *README.md* in the project directory) via its *MOVE-PUB* socket.
- After the replica reaches the maximum Lamport timestamp, the main thread associated with the process corresponding to the replica will periodically broadcast (for instance, every 0.3 seconds) `DONE` messages, to inform the other replicas that it no longer generates any `Move` operations, again, with its *MOVE-PUB* socket to the "DONE" topic.
- Meanwhile, the listener thread of the replica will be polling its *MOVE-SUB* socket, which will be subscribed to the topics "MOVE" and "DONE". If it receives a message with the "MOVE" topic, that is, the message describes a `Move` operation, then it will not broadcast any messages in exchange, only applying the received operation.
- However, if the listener thread receives a `DONE` message, which means there is an incoming message with the topic "DONE", then it is going to broadcast an `ACK` message, through the *ACK-PUB* socket, corresponding to the `DONE` message it received, according to the message format which will be explained later.
- The `ACK` messages coming from other replicas, meanwhile, are polled for by the main thread, via the *ACK-SUB* socket.

The message formats, on the other hand, are the following, with respect to the topics they are published to:

- The message format for the topic "MOVE":

```
{
    'sender_id': int, # The ID of the replica
                        which generates and sends
                        the Move operation
    'timestamp': int, # The Lamport timestamp
                        associated with the
                        Move operation
    'parent': int, # p for the Move operation
    'metadata': dict, # m for the Move operation
    'child': int # c for the Move operation
}
```

- The message for the topic "DONE":

```
{
    'sender_id': int, # The ID of the replica
                        which sends the DONE message
    'timestamp': int # The current Lamport timestamp
                       of the sending replica
}
```

- The message format for the topic "ACK":

```
{
    'sender_id': int, # The ID of the replica
                        which sends the ACK
    'timestamp': int, # The current Lamport timestamp
                        of the sending replica
    'ack_to': int, # The ID of the replica
                     which sent the DONE message
    'ack_timestamp': int # The timestamp received from
                           the other replica for
                           the DONE message
}
```

Hint: You can use the `send_multipart()` method provided by ZeroMQ for the ZeroMQ sockets to send the topic, which is going to be encoded into the built-in Python type `bytes`, and the message content, which you are going to encode into a **JSON object**, in separate buffers at the same time.

3 The Main Program

The main program for the system, which is in the file *main.py*, has three primary responsibilities, partitioned across three functions. The `main(num_replicas, tree_config, hosts, main_base, listener_base, max_timestamp)` function, executed by the main process, is responsible for creating and handling the replica processes; the `run_replica(run_id, tree_config, replica_id, replica_info,`

num_replicas, hosts, max_timestamp) function, executed by the main threads of the replica processes, is responsible for setting up the replica, creating and handling the listener thread, and broadcasting Move operations and DONE messages to other replicas, while receiving ACK messages for the latter type of messages, i.e., the DONE messages, it sends from other replicas.

You are provided with a partially completed version of *main.py* in the project directory. In order to have a proper implementation, you will need to complete the three functions described above.

3.1 Main Process of the System: main(num_replicas, tree_config, hosts, main_base, listener_base, max_timestamp)

In the function main(num_replicas, tree_config, hosts, main_base, listener_base, max_timestamp), the main process of the system first creates a random UUID for the current run, so that the final state of the tree could be written into unique files by the replicas at the end, and handles the creation of the *runs* directory in case that it does not exist.

In addition to the above functionalities, which are already implemented for you, you are going to create the replica processes for the replicas which you are going to run locally (see *README.md* for more information on specifying local and remote replicas) using the multiprocessing.Process class. The target for the replica processes is going to be the run_replica function, with the arguments (run_id, tree_config, id, (hosts[id], main_base, listener_base), num_replicas, hosts, max_timestamp). After creating the replica processes, you are going to start these processes using the appropriate method provided by the multiprocessing module, and then, you are going to join these processes in the same fashion.

3.2 Main Thread of a Replica: run_replica(run_id, tree_config, replica_id, replica_info, num_replicas, hosts, max_timestamp)

In the function run_replica(run_id, tree_config, replica_id, replica_info, num_replicas, hosts, max_timestamp), the main thread of the replica will first create the Replica object for the replica with the arguments from replica_id and replica_info, and the threading.Event objects shutdown_event, which is going to be used as the event that the replica is shutting down, and all_replicas_done_event, which is going to be invoked when all replicas are found out to be DONE. Then, it will create and set up the ZeroMQ context, and the *MOVE-PUB* and *ACK-SUB* ZeroMQ sockets for the replica.

Later, the main thread will create the listener thread for the replica, with the target as a new function with the arguments (replica_obj, zmq_context, shutdown_event, replica_info, num_replicas, hosts, all_replicas_done_event), and start the listener thread.

After completing the setup for the listener thread, the main thread will execute a loop which will continue until the maximum timestamp is reached;

in that loop, it will generate `Move` operations using the configuration generators (see *README.md* and *main.py* for details), apply these operations on the `Replica` object by invoking `apply_local_move(parent, metadata, child)`, and broadcast the data present in the returned `MovePayload` instance (see *src/tree_crdt/payload/move.py*) in a `Move` message to other replicas.

When the replica reaches the maximum timestamp, the main thread will begin to broadcast `DONE` messages, periodically, to all of the replicas, and it will also poll its *ACK-SUB* socket for `ACK` messages. Upon receiving an `ACK` message, the main thread will check whether the target for the message is itself, via the field 'ack_to', and whether the message is not a self-message, that is, the sending replica is not the same replica as the receiving one, through the field 'sender_id', after extracting the JSON object from the message content. Using the information from the two mentioned fields, the main thread will keep track of the replicas that have sent an `ACK` message in exchange for the `DONE` messages sent by the replica; if all replicas are found to have sent `ACK` messages to the replica, then the main thread will set `shutdown_event`, and join the listener thread.

At the end, the main thread prints the final state of the tree, along with the maximum timestamp and the tree configuration, to a TXT file (see *README.md* and the directory *sample_runs* for some examples). Please note that the printing functionality is already implemented in *main.py*, and you do not need to perform any modifications regarding this functionality.

3.3 Listener Thread of a Replica: Function with `(replica_obj, zmq_context, shutdown_event, replica_info, num_replicas, hosts, all_replicas_done_event)`

In the function you are going to create with the arguments specified above, the listener thread will first create and set up the *MOVE-SUB* and *ACK-PUB* sockets for the replica. Then, while `shutdown_event` is not set, it will poll the *MOVE-SUB* socket for `Move` operations and `DONE` messages. Upon receiving a message, the listener thread will extract the content into a string comprising the topic, and a JSON object containing the actual content according to the message formats specified before.

If the topic of the received message is "MOVE", then the listener thread will extract the necessary fields for the corresponding `Move` operation, create the `MovePayload` instance, and apply the operation by invoking `apply_remote_move(op)` with the `MovePayload` instance on `replica_obj`.

On the other hand, if the topic is "DONE" for the received message, the listener thread will extract the message content into a JSON object, and will retrieve the ID and Lamport timestamp of the sender replica from that object. It will check whether the message is not a self-message, and if not, it will update the Lamport clock of the replica with the received Lamport timestamp, prepare the `ACK` message targeting the replica which sent the `DONE` message, which is going to be indicated through the field 'ack_to', and broadcast the `ACK` message to other replicas.

In addition to these functionalities, the listener thread will also keep track of the replicas from which `DONE` messages are received. If all replicas are found to be broadcast `DONE` messages, that is, if the listener thread has received `DONE` messages from all the other replicas, then it will set `all_replicas_done_event`, which will cause the main thread to cease broadcasting `DONE` messages and continue with setting `shutdown_event` and the subsequent termination of the replica process.

4 The `tree-crdt` Library

The `tree-crdt` library is responsible for holding the data types and structures related to the Tree CRDT and its components, namely, the conceptualization of a replica as an object, the tree data structure that underlies the Tree CRDT, a node representation for the nodes of the tree, and the logical clock that the replicas are going to use.

You are currently provided with a partially completed version of the `tree-crdt` library in the directory *src/tree_crdt*. In order to complete the library, you need to implement the classes described below.

4.1 `LamportClock` Class

Residing at *src/tree_crdt/clock/lamport.py*, the `LamportClock` class is a class created for storing the Lamport clock of a replica as a Python object. You should be writing your implementation for this class as a concrete implementation of the `Clock` abstract class (see *src/tree_crdt/clock/clock.py*).

The fields of the class should be the following:

- `__id`: The ID of the replica which owns the `LamportClock` instance. It should have an integer value.
- `__timestamp`: The current Lamport timestamp of the `LamportClock` instance. It should have an integer value.

Furthermore, the following abstract methods should be implemented in the `LamportClock` class:

- `__init__(self, id)`: The constructor for the `LamportClock` class. You will need to set the value of `__id` to the value provided in the parameter `id`, and the value of `__timestamp` to 0.
- `@property id(self)`: Returns the value of the field `__id`.
- `@property timestamp(self)`: Returns the value of the field `__timestamp`.
- `update(self, received)`: Updates the Lamport timestamp according to the received value, which can be either `None` in case of a local update, or an integer value in the case of updating with respect to a remote operation.
- `__str__(self)`: Returns the string representation of the field `__timestamp`.

4.2 `Tree` Class

Residing at *src/tree_crdt/tree/tree.py*, the `Tree` class is where the underlying tree data structure for the Tree CRDT is held. For this class, you are expected to implement the following methods in addition to storing the tree state in a container of `Node` objects (see *src/tree_crdt/tree/node.py*):

- `__init__(self)`: The constructor for the `Tree` class. You will need to set the container for the `Node` objects into an empty container of the type you select to use.
- `__call__(self)`: Overrides the `()` operator, which calls the object on which it is invoked (If your object is named `tree`, then you can use this operator as `tree()`), for the `Tree` class. Returns the current state of the tree according to the set representation described before.
- `__getitem__(self, key)`: Overrides the index operator, `[]`, for the `Tree` class. Given the ID of a node, returns the associated `Node` object if it exists in the tree, or `None` otherwise.
- `move(self, new_node)`: The method through which you will apply the `Move` operation **within the data structure**, via invocations in a function belonging to another class. Specifically, in this function, you are going to check whether `new_node` already exists in the tree with the exact components p, m, and c which you provide with `new_node`. If the node is not preexisting in the tree, then you will check whether moving `new_node` creates a cycle; if it does not create a cycle, you can safely move `new_node` under its new parent, with the metadata you gave; otherwise, as stated before, you are going to ignore the node and keep the structure of the tree intact.
  - **Note:** If you already have (p', m', c) with $p' \neq p \vee m' \neq m$ in your tree, then you must ensure that the entry for (p', m', c) has been removed from the tree.
- `__str__(self)`: Returns the string representation of the tree state; you can check the *sample_runs* directory for the representation.

Note: You can implement additional helper methods if you need in the `Tree` class.

4.3 `Replica` Class

Residing at *src/tree_crdt/replica.py*, the `Replica` class is the class in which the important properties of a replica, such as the logical clock, the tree, the operation log, and the communication addresses, are stored.

An important point regarding the `Replica` class is that the `Replica` instance in a replica process is shared between the main and listener threads of the process, which causes the class to be a concurrent data structure. Therefore, the accesses and updates to the clock, tree, and operation log of a `Replica` instance must be implemented in a way that ensures the thread-safety of such actions.

Namely, the class should have the following fields:

- `__id`: The ID of the replica. It should have an integer value.
- `__clock`: The `Clock` instance storing the logical clock associated with the replica.
  - **Note:** Even though in this phase you are going to store a `LamportClock` instance as the `Clock` instance, that requirement may change in the future phases of the project.
- `__tree`: The `Tree` instance storing the tree data structure of the Tree CRDT for the replica.
- `__op_log`: A list of $(\text{replica_id}, t, \text{old_}p, p, m, c)$ tuples that you should keep as the log of `Move` operations applied so far on the replica.
- `__zmq_main_addr`: The address for the main thread of the replica. It should have the following value: `f'tcp://{host}:{main_base + id}'`.
- `__zmq_listener_addr`: The address for the listener thread of the replica. It should have the following value: `f'tcp://{host}:{listener_base + id}'`.
- **Note:** Since a `Replica` object is a concurrent data structure, you will also need to embrace a locking strategy inside the functions that involve accessing or updating `__clock`, `__tree`, or `__op_log`. You are free to implement any locking strategy you wish, whether coarse- or fine-grained, as long as your implementation is correct.

Moreover, in the `Replica` class, you should implement the following methods at a minimum (you can implement additional helper methods if you need):

- `__init__(self, id, host, main_base, listener_base)`: The constructor for the `Replica` class. You will need to
- `@property id(self)`: Returns the value of the field `__id`.
- `@property clock(self)`: Returns a deep copy of the value of the field `__clock`.
  - **Hint:** You can check the functions provided by the standard Python module `copy`.
- `@property tree(self)`: Returns a deep copy of the value of the field `__tree`.
- `@property log(self)`: Returns a deep copy of the value of the field `__op_log`.
- `@property main_addr(self)`: Returns the value of the field `__zmq_main_addr`.
- `@property listener_addr(self)`: Returns the value of the field `__zmq_listener_addr`.

- `current_timestamp(self)`: Returns the value of the current timestamp for the `Clock` instance of the replica.
  - **Hint:** You may want to access the `Clock` instance itself here, rather than a copy for it.
- `tick_clock(self, received)`: Invokes the `update(received)` method of `__clock` in a thread-safe manner, and returns the timestamp of `__clock`. Use this method when you are going to update your Lamport timestamp upon reception of a `DONE` message.
- `__apply_move(self, op)`: Accepting a `MovePayload` instance as its parameter, this method is where you are going to implement the detailed logic of the `Move` operation on a replica using the `Tree` data structure. Precisely, you are first going to check, using the operation log you keep in the field `__op_log`, whether the last operation in the log is a predecessor of the operation described in `op` with respect to the ordering explained in Subsection 1.3.

  If the operation in `op` is a successor of the last operation in the log, then you are going to *do* (apply) the operation directly, inserting the information regarding the operation, as a tuple $(\text{replica_id}, t, \text{old_}p, p, m, c)$, into the operation log and invoking `move(new_node)` with the `Node` you create with the given parameters from `op` on `__tree`.

  If the operation in `op` is a predecessor of the last operation in the log, however, then you are going to apply the *undo-do-redo* sequence as explained before, retrieving the operations to be undone and subsequently redone from the operation log in the meantime.
- `apply_local_move(self, parent, metadata, child)`: Takes the ID for the parent node, the dictionary for the metadata, and the ID for the child node as its parameters. Updates the clock of the replica for the local operation, creates a `MovePayload` instance with the ID and the Lamport timestamp of the replica, along with `parent`, `metadata`, and `child`, and applies the local operation for the payload by invoking `__apply_move(op)`; at the end of its execution, returns the created `MovePayload` instance. Use this method to generate and apply a local `Move` operation an on a replica.
- `apply_remote_move(self, op)`: Taking a `MovePayload` instance, which you are going to create with the contained information when you receive a remote `Move` operation from another replica, this function is the one you should invoke when applying a remote `Move` operation on a replica. It updates the Lamport clock of the replica with respect to the timestamp of the operation described in `op`, and calls `__apply_move(op)` to apply the operation on the replica.
- `__str__(self)`: Returns the string representation of a replica: `f'ID: {self.__id}, Timestamp: {self.current_timestamp()}'`

5 Package Contents

Check the file *README.md* in the root directory of the project for a detailed description on the package contents, the project setup and how to run the project.

One particular type of content you may want to check is the *sample_runs* directory; it contains different example cases for running a correct implementation, and you may benefit from it for formatting purposes regarding the string representations and related output.

- **Note:** Please note that the numbers of nodes in the final state of the tree may differ between different executions of the system with the same configuration; **this situation does not imply any issue with your implementations**.

6 Submission Guideline

The submissions you are going to make must include the following files and directories:

- The *src* directory under which you have completed your implementations for the `tree-crdt` library.
- The *main.py* file which you have implemented as the main program.

Place the files/directories listed above into a single ZIP archive named *CS_403-534_Project_Phase_1_Group_yourgroupnumber.zip*, and submit it under the assignment for `Project - Phase 1` on SUCourse+.

Please also note that a single submission per group is sufficient.

7 Grading Criteria

- **Lamport Clock Implementation (15 pts):** Correct Lamport clock behavior, proper compatibility with the `Clock` abstract class.
- **Tree CRDT Semantics (25 pts):** Correct `Move` semantics and implementation, preservation of the tree structure against invalid operations.
- **Replica Ordering and Convergence (35 pts):** Proper handling for in- and out-of-order operations (including a correct ordering), convergence to equivalent final state across replicas.
- **Runtime Orchestration in `main.py` (15 pts):** Proper process orchestration per replica, proper running of the replica processes on the function `run_replica()` with graceful shutdown behavior.
- **Robustness and Error Handling (10 pts):** Reliable behavior under edge cases and malformed operations, with preserved tree/log invariants and thread-safe concurrent access.
- **BONUS (10 pts):** We currently know that there exists an erroneous case with the communication protocol such that replica(s) may fail to shut down. If you would like to get a bonus grade from Phase 1, you should determine the exact conditions for the case to occur, explain the case in a 1-page PDF document, and if possible, try to come up with a correct solution.